

[← Go Back](#)

Hardware

Nicla Sense ME



Tutorials

Arduino Nicla Sense ME Cheat Sheet

Sensors Readings on a
Local Webserver

Connecting the Nicla Sense
ME to IoT Cloud

Getting Started with Nicla
Sense ME

Arduino Nicla Sense ME as
a MKR Shield

Nicla Sense ME User
Manual

Displaying on-Board Sensor
Values on a WebBLE
Dashboard

[Home](#) / [Hardware](#) / [Nicla Sense ME](#) / **Arduino Nicla Sense ME Cheat Sheet**

ON THIS PAGE

Arduino Nicla Sense ME Cheat Sheet

Learn how to set up the Arduino Nicla Sense ME and get a quick overview of the components. Obtain information regarding pins and how to use the different sensors.

Author · Sebastian Romero

Last revision · 07/17/2024

Goals

Core

Installation

Pins

RGB LED

Sensors

BSX Sensor Fusion Softwar

Communication

Conclusion



The Arduino Nicla Sense ME

The Arduino® Nicla Sense ME is our smallest form factor yet, with a range of industrial grade sensors packed into a tiny footprint. It features 4 industrial grade Bosch sensors that can accurately measure rotation, acceleration, pressure, humidity, temperature, air quality and CO2 levels.

Goals

This article is a collection of guides, API calls, libraries and tutorials that can help you get started with the Nicla Sense ME board. You can also visit the [documentation product page for the Nicla Sense ME](#).

Core

The Nicla Sense ME uses the [Arduino Mbed OS Nicla Boards core](#).

Installation

Arduino IDE 1.8.X

The Nicla Sense ME can be programmed through the **Classic Arduino IDE 1.8.X**. To install your board, you can check out the guide below:

[Installing the Arduino Mbed OS Nicla Boards core](#)

Arduino IDE 2

The Nicla Sense ME can be programmed through the **Arduino IDE 2**. To install your board, you can check out the guide below:

[How to use the board manager with the Arduino IDE 2](#)

Cloud Editor

The Nicla Sense ME can be programmed through the **Cloud Editor**. To

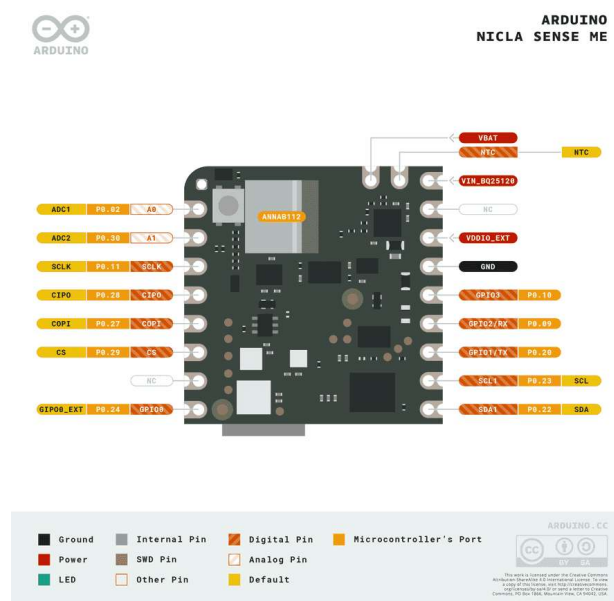
Getting started with the Cloud Editor

Board Not Detected

Sometimes the board is not detected even when the board is connected to your computer. This can be solved through the following steps:

1. Disconnect the board from your computer by removing the USB cable.
2. Reconnect the board to your computer.
3. If it still doesn't show up, double-tap the reset button, to activate the bootloader mode.

Pins



The pinout for Nicla Sense ME.

Analog Pins

The Nicla Sense ME has 2 analog pins (A0 and A1), that can be used through the `analogRead()` function.

```
1 int value = analogRead(pin);
```

PWM Pins

Most of the digital and analog pins can be used as PWM (Pulse Width

```
1 analogWrite(pin, value);
```

Digital Pins

There are a total of 10 digital pins, whereas the 2 analog pins can also be used as digital pins.

To use them, they need to be configured. Usually this is done in the `setup()` function.

```
1 pinMode(pin, INPUT); //configured as an input
2 pinMode(pin, OUTPUT); //configured as an output
3 pinMode(pin, INPUT_PULLUP); //uses an internal pull
```

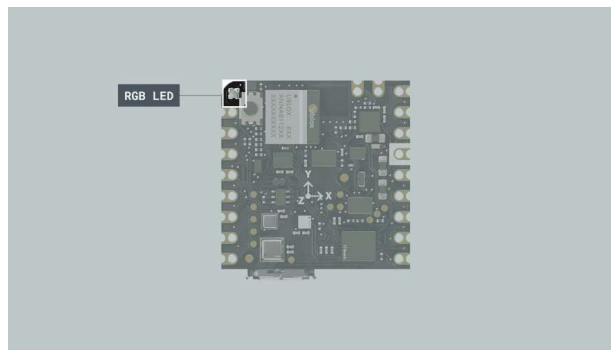
To read the state of a digital pin:

```
1 state = digitalRead(pin);
```

To write a state to a digital pin:

```
1 digitalWrite(pin, HIGH);
```

RGB LED



The RGB LED is located in the rounded corner

The Nicla Sense ME features a built-in RGB that can be utilized as a feedback component for applications. The LED is connected through

The Nicla System header is required to use the RGB LED.

```
1 #include "Nicla_System.h"
```

Since the functions are scoped under a specific Class name called "nicla", you need to explicitly write it before each statement.

The LEDs need to be started along with the Nicla inside

```
void setup() {
```

```
1 nicla::begin();  
2 nicla::leds.begin();
```

The LED can be set to the desired RGB value using red, green and blue components or by using one of the following predefined colors:

- off
- red
- green
- blue
- yellow
- magenta
- cyan

To set the LED to a predefined color (e.g. green or blue):

```
1 void loop() {  
2   nicla::leds.setColor(green);  
3   delay(1000);  
4   nicla::leds.setColor(blue);  
5   delay(1000);  
6 }
```

To turn the LED off:

```
1 nicla::leds.setColor(off);
```

You can also choose a value between 255 - 0 for each color component to set a custom color:

```

1 void loop() {
2   int red = 234;
3   int green = 72;
4   int blue = 122;
5
6   nicla::leds.setColor(red, green, blue);
7   delay(1000);
8   nicla::leds.setColor(off);
9   delay(1000);
10 }

```

This is a complete example code to blink the built-in I2C LED:

```

1 #include "Nicla_System.h"
2
3 void setup() {
4   nicla::begin();
5   nicla::leds.begin();
6 }
7
8 void loop() {
9   nicla::leds.setColor(red);
10  delay(1000);
11  nicla::leds.setColor(off);
12  delay(1000);
13 }

```

Sensors

There are three ways to read from the on-board sensors:

- 1 Read the sensors directly from Nicla Sense ME in standalone mode.
- 2 Read sensor values through Bluetooth® Low Energy
- 3 Read sensor values through I2C by connecting an ESLOV cable

To read from the sensors in any of these mode, you need to install the **Arduino_BHY2** and **Arduino_BHY2Host** libraries. These can be found in the library manager using the Arduino IDE. To do so in the IDE, select **Tools->Manage Libraries...**, now search for **Arduino_BHY2** and **Arduino_BHY2Host** in the new window that opened and click on the install button.

To use the sensors in your sketches, you need to know the sensors ID. You can find them in the section "Sensor IDs" of this article. They can also be found in the header file [here](#). Additionally, there is an example sketch in the library that will print all available sensors in the Serial Monitor of the Arduino IDE. This example sketch can be found in **File > Examples > Arduino_BHY2 > ShowSensorList** in the Arduino IDE.

In the following section, you can see how these ID's are used in an

Sensor Classes

Sensor: This class handles all the other sensors which have a single value to be read, like temperature, gas, pressure, etc. And also the event sensors, like the step detector. This generic sensor class provides the sensor data through the `value` property that returns a **float**.

SensorOrientation: This class handles sensors with the Euler format, used for example with orientation. It allows you to read the `pitch`, `roll` and `heading` property.

SensorXYZ: This class handles sensors with the XYZ format, like the accelerometer and the gyroscope. It contains `x`, `y` and `z` values

SensorQuaternion: Can be used to handle sensors with the quaternion format, can be used to calculate rotation vector, game rotation vector and geomagnetic rotation vector. You can access the `x`, `y`, `z` and `w` property using this class.

SensorActivity: Use this class to handle sensors with the activity format. The activity is encoded as ID and can be retrieved from the `value` property. Use `getActivity` to get a human readable version of the activity e.g. "Walking activity started".

SensorBSEC: BSEC stands for Bosch Sensortec Environmental Cluster, basically you can access the air quality (IAQ) level and it contains the following data:

Function	Description	Data type
<code>iaq()</code>	IAQ value for regular use case	unsigned 16bit
<code>iaq_s()</code>	IAQ value for stationary use cases	unsigned 16bit
<code>b_voc_eq()</code>	breath VOC equivalent (ppm)	float
<code>co2_eq()</code>	CO2 equivalent (ppm) [400,]	unsigned 32bit
<code>comp_t()</code>	compensated temperature (Celsius)	float
<code>comp_h()</code>	compensated humidity	float
<code>comp_g()</code>	compensated gas resistance (Ohms)	unsigned 32bit
<code>accuracy()</code>	accuracy level: [0-3]	unsigned 8bit

Further down you can see how these objects are used in an Arduino sketch to get readings from the sensors.

Sensor IDs

The IDs to address the sensors both through ESLOV and WebBLE are as follows:

ID	Description	SENSOR_ID MACRO	Class
1	Accelerometer	SENSOR_ID_ACCELEROMETER	SensorXYZ

ID	Description	SENSOR_ID MACRO	Class
3	Accelerometer uncalibrated	SENSOR_ID_ACC_RAW	SensorXYZ
4	Accelerometer corrected	SENSOR_ID_ACC	SensorXYZ
5	Accelerometer offset	SENSOR_ID_ACC_BIAS	SensorXYZ
6	Accelerometer corrected wake up	SENSOR_ID_ACC_WU	SensorXYZ
7	Accelerometer uncalibrated wake up	SENSOR_ID_ACC_RAW_WU	SensorXYZ
10	Gyroscope passthrough	SENSOR_ID_GYRO_PASS	SensorXYZ
12	Gyroscope uncalibrated	SENSOR_ID_GYRO_RAW	SensorXYZ
13	Gyroscope corrected	SENSOR_ID_GYRO	SensorXYZ
14	Gyroscope offset	SENSOR_ID_GYRO_BIAS	SensorXYZ
15	Gyroscope wake up	SENSOR_ID_GYRO_WU	SensorXYZ
16	Gyroscope uncalibrated wake up	SENSOR_ID_GYRO_RAW_WU	SensorXYZ
19	Magnetometer passthrough	SENSOR_ID_MAG_PASS	SensorXYZ
21	Magnetometer uncalibrated	SENSOR_ID_MAG_RAW	SensorXYZ
22	Magnetometer corrected	SENSOR_ID_MAG	SensorXYZ
23	Magnetometer offset	SENSOR_ID_MAG_BIAS	SensorXYZ
24	Magnetometer wake up	SENSOR_ID_MAG_WU	SensorXYZ
25	Magnetometer uncalibrated wake up	SENSOR_ID_MAG_RAW_WU	SensorXYZ
28	Gravity vector	SENSOR_ID_GRA	SensorXYZ
29	Gravity vector wake up	SENSOR_ID_GRA_WU	SensorXYZ
31	Linear acceleration	SENSOR_ID_LACC	SensorXYZ

ID	Description	SENSOR_ID MACRO	Class
	wake up		
34	Rotation vector	SENSOR_ID_RV	SensorQuaternion
35	Rotation vector wake up	SENSOR_ID_RV_WU	SensorQuaternion
37	Game rotation vector	SENSOR_ID_GAMERV	SensorQuaternion
38	Game rotation vector wake up	SENSOR_ID_GAMERV_WU	SensorQuaternion
40	Geomagnetic rotation vector	SENSOR_ID_GEORV	SensorQuaternion
41	Geomagnetic rotation vector wake up	SENSOR_ID_GEORV_WU	SensorQuaternion
43	Orientation	SENSOR_ID_ORI	SensorOrientation
44	Orientation wake up	SENSOR_ID_ORI_WU	SensorOrientation
48	Tilt detector	SENSOR_ID_TILT_DETECTOR	Sensor
50	Step detector	SENSOR_ID_STD	Sensor
52	Step counter	SENSOR_ID_STC	Sensor
53	Step counter wake up	SENSOR_ID_STC_WU	Sensor
55	Significant motion	SENSOR_ID_SIG	Sensor
57	Wake gesture	SENSOR_ID_WAKE_GESTURE	Sensor
59	Glance gesture	SENSOR_ID_GLANCE_GESTURE	Sensor
61	Pickup gesture	SENSOR_ID_PICKUP_GESTURE	Sensor
63	Activity recognition	SENSOR_ID_AR	SensorActivity
67	Wrist tilt gesture	SENSOR_ID_WRIST_TILT_GESTURE	Sensor
69	Device orientation	SENSOR_ID_DEVICE_ORI	SensorOrientation
70	Device orientation wake up	SENSOR_ID_DEVICE_ORI_WU	Sensor
75	Stationary detect	SENSOR_ID_STATIONARY_DET	Sensor
77	Motion detect	SENSOR_ID_MOTION_DET	Sensor
91	Accelerometer offset wake up	SENSOR_ID_ACC_BIAS_WU	SensorXYZ

ID	Description	SENSOR_ID MACRO	Class
92	Gyroscope offset wake up	SENSOR_ID_GYRO_BIAS_WU	SensorXYZ
93	Magnetometer offset wake up	SENSOR_ID_MAG_BIAS_WU	SensorXYZ
94	Step detector wake up	SENSOR_ID_STD_WU	Sensor
115	BSEC data	SENSOR_ID_BSEC	SensorBSEC
128	Temperature	SENSOR_ID_TEMP	Sensor
129	Barometer	SENSOR_ID_BARO	Sensor
130	Humidity	SENSOR_ID_HUM	Sensor
131	Gas	SENSOR_ID_GAS	Sensor
132	Temperature wake up	SENSOR_ID_TEMP_WU	Sensor
133	Barometer wake up	SENSOR_ID_BARO_WU	Sensor
134	Humidity wake up	SENSOR_ID_HUM_WU	Sensor
135	Gas wake up	SENSOR_ID_GAS_WU	Sensor
136	Hardware Step counter	SENSOR_ID_STC_HW	Sensor
137	Hardware Step detector	SENSOR_ID_STD_HW	Sensor
138	Hardware Significant motion	SENSOR_ID_SIG_HW	Sensor
139	Hardware Step counter wake up	SENSOR_ID_STC_HW_WU	Sensor
140	Hardware Step detector wake up	SENSOR_ID_STD_HW_WU	Sensor
141	Hardware Significant motion wake up	SENSOR_ID_SIG_HW_WU	Sensor
142	Any motion	SENSOR_ID_ANY_MOTION	Sensor
143	Any motion wake up	SENSOR_ID_ANY_MOTION_WU	Sensor

The syntax to instantiate a sensor object is

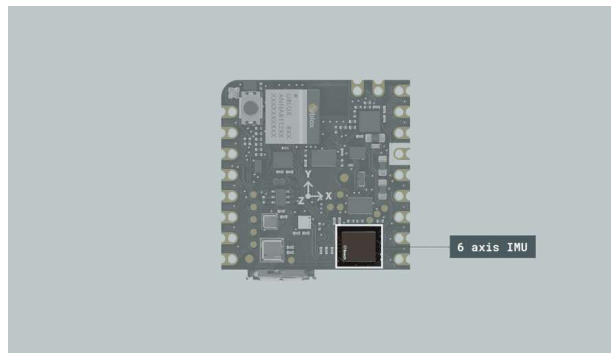
`SensorClass variableName(SENSOR_ID_MACRO)`. For example:

```
1 // Declaring Accelerometer uncalibrated
2 SensorXYZ accelerometerRaw(SENSOR_ID_ACC_RAW);
```

Standalone Mode

In standalone mode, the sensors can be accessed through sensor objects. The sensor data is then read by Nicla Sense ME's on-board microcontroller.

IMU



The IMU sensor

Follow these steps to use the library to read the sensor values.

Include the library's header file:

```
1 #include "Arduino_BHY2.h"
```

Define sensor objects:

```
1 SensorXYZ accelerometer(SENSOR_ID_ACC);
2 SensorXYZ gyro(SENSOR_ID_GYRO);
```

Activating the sensors:

```
1 void setup(){
2   Serial.begin(115200);
3   BHY2.begin();
4
5   accelerometer.begin();
6   gyro.begin();
7 }
```

The `begin()` function starts the sensor by calling the `configure()` with default parameters, making it easy to start and use on-board sensors. The parameters in the `configure()` function are sample rate and latency. If specific parameters are needed, then simply call `configure()` with your preferred values. E.g.: `configure(10, 1)`. In this case, the sample rate would be set to `10 Hz` and the latency would be `1ms`.

Sample rate is used also to enable/disable the sensor. 0 to disable, > 0 to enable.

Latency is the longest delay, in milliseconds, before the host is notified of a new value from a sensor. Even though the latency parameter is a 32-bit integer, only the least significant 24 bits of it are actually used. A latency of 0 means that the host is notified immediately when a new value is available.

Reading the sensor values:

```
1 void loop(){
2   static auto lastCheck = millis();
3
4   // Update function should be continuously polled
5   BHY2.update();
6
7   // Check sensor values every second
8   if (millis() - lastCheck >= 1000) {
9     lastCheck = millis();
10
11     Serial.println(String("acceleration: ") + accel
12     Serial.println(String("gyroscope: ") + gyro.toS
13   }
14 }
```

You can read the numeric x, y and z properties of the accelerometer and the gyroscope as follows:

```
1 short accX = accelerometer.x();
2 short accY = accelerometer.y();
3 short accZ = accelerometer.z();
4
5 short gyroX = gyroscope.x();
6 short gyroY = gyroscope.y();
7 short gyroZ = gyroscope.z();
```

Temperature

To read the temperature in standalone mode, you also need to use the `Arduino_BHY2` library as described in the section above.

Follow these steps to use the library to read the sensor values.

1. Include the `BHY2` header files:

```
1 #include "Arduino_BHY2.h"
```

Define the sensor object:

```
1 Sensor temperature(SENSOR_ID_TEMP);
```

Activating the sensor:

```
1 void setup(){
2   Serial.begin(115200);
3   BHY2.begin();
4   temperature.begin();
5 }
```

Reading the sensor value:

```
1 void loop(){
2   static auto lastCheck= millis();
3   BHY2.update();
4
5   // Check sensor values every second
6   if (millis() - lastCheck >= 1000) {
7     lastCheck = millis();
8     Serial.println(String("temperature: ") + String
9   }
10 }
```

Gas

To get readings from the gas sensor in standalone mode, you also need to use the Arduino_BHY2 library as described in the section above.

Follow these steps to use the library to read the sensor values.

Include the library's header file:

```
1 #include "Arduino_BHY2.h"
```

Define the sensor object:

```
1 Sensor gas(SENSOR_ID_GAS);
```

Activating the sensor:

```
1 void setup() {  
2   Serial.begin(115200);  
3   BHY2.begin();  
4   gas.begin();  
5 }
```

Reading the sensor value:

```
1 void loop(){  
2   static auto lastCheck= millis();  
3   BHY2.update();  
4  
5   // Check sensor values every second  
6   if (millis() - lastCheck >= 1000) {  
7     lastCheck = millis();  
8     Serial.println(String("gas: ") + String(gas.val  
9   }  
10 }
```

Pressure

To get readings from the pressure sensor in standalone mode, you also need to use the Arduino BHY2 library as described in the section above.

Follow these steps to use the library to read the sensor values in hPa.

Include the library's header file:

```
1 #include "Arduino BHY2.h"
```

Define the sensor object:

```
1 Sensor pressure(SENSOR_ID_BARO);
```

Activating the sensor:

```
1 void setup() {
2   Serial.begin(115200);
3   BHY2.begin();
4   pressure.begin();
5 }
```

Reading the sensor value:

```
1 void loop(){
2   static auto lastCheck= millis();
3   BHY2.update();
4
5   // Check sensor values every second
6   if (millis() - lastCheck >= 1000) {
7     lastCheck = millis();
8     Serial.println(String("pressure: ") + pressure.);
9   }
10 }
```

Quaternion Rotation

To get readings from the IMU in a quaternion format in standalone mode, you also need to use the Arduino BHY2 library as described in the section above.

Follow these steps to use the library to read the sensor values.

Include the library's header file:

```
1 #include "Arduino BHY2.h"
```

Define the sensor object:

```
1 SensorQuaternion rotation(SENSOR_ID_RV);
```

Activating the sensor:

```
1 void setup(){
2   Serial.begin(115200);
3
4   BHY2.begin();
5   rotation.begin();
6 }
```

```
1 void loop(){
2   static auto lastCheck = millis();
3   BHY2.update();
4
5   if (millis() - lastCheck >= 1000) {
6     lastCheck = millis();
7     Serial.println(String("rotation: ") + rotation.t
8   }
9 }
```

Activity

To get activity status in standalone mode, you also need to use the Arduino BHY2 library as described in the section above.

Follow these steps to use the library to read the sensor values.

Include the library's header file:

```
1 #include "Arduino BHY2.h"
```

Define the sensor object:

```
1 SensorActivity activity(SENSOR_ID_AR);
```

Activating the sensor:

```
1 void setup(){
2   Serial.begin(115200);
3
4   BHY2.begin();
5   activity.begin();
6 }
```

Reading the sensor value:

```
1 void loop(){
2   static auto lastCheck = millis();
3   BHY2.update();
4
5   if (millis() - lastCheck >= 1000) {
6     printTime = millis();
7     Serial.println(String("Activity info: ") + activ
8   }
9 }
```

BSEC Data

To get readings from the BME sensor in standalone mode, you also need to use the Arduino_BHY2 library as described in the section above.

Follow these steps to use the library to read the sensor values.

Include the library's header file:

```
1 #include "Arduino_BHY2.h"
```

Define the sensor object:

```
1 SensorBSEC bsec(SENSOR_ID_BSEC);
```

Activating the sensor:

```
1 void setup(){
2   Serial.begin(115200);
3
4   BHY2.begin();
5   bsec.begin();
6 }
```

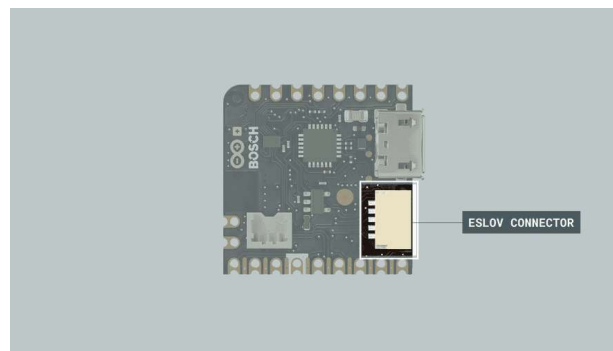
Reading the sensor value:

```

1 void loop(){
2   static auto lastCheck = millis();
3   BHY2.update();
4
5   if (millis() - lastCheck >= 1000) {
6     printTime = millis();
7     Serial.println(String("BSEC info: ") + bsec.toSt
8   }
9 }

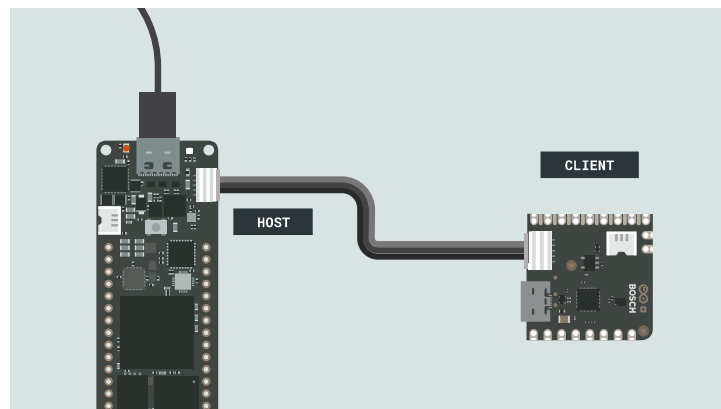
```

Sensor Data Over ESLOV



The ESLOV Connector is located next to the USB port

In order to transmit data over ESLOV to another Arduino board you need to connect the boards with an ESLOV cable. The Nicla Sense ME could for example be connected to an Arduino Portenta H7.



- 1 To have the Nicla Sense ME pass the sensor data through ESLOV, you need to upload the **App** sketch. You can find it in the Examples menu in the IDE under **Arduino_BH2 > App**. After you upload the sketch, you can disconnect the Nicla Sense ME from the USB cable. It can be powered through the ESLOV connection.
- 2 For the receiving device you need to upload the passthrough sketch. You can find it in the Examples menu in the IDE under **Arduino BHY2Host > Passthrough**. After you upload

through the sensor data. The latter one is the one you will use to configure the sensors and read from them.

- 3 When you're done uploading the sketches, you can use the **bhy** script to interact with the sensors. In the downloaded package (see [here](#)), navigate into the tools folder. There you will find binaries of the bhy tool for Linux and Windows. If you like, you can build the tool yourself (e.g. if you are on macOS). For that, if you haven't installed **Go** yet, please do so by following [there](#) instructions. From the terminal execute this command to start the build: `go build`
- 4 Use the bhy command as follows:

```

1 # list available serial ports
2 ./bhy list
3
4 # read available sensor data
5 ./bhy sensor read -p /dev/ttyACM2
6
7 # continuously read sensor data when available
8 ./bhy sensor read -live -p /dev/ttyACM2
9
10 # configure sensor 10 with a sample rate of 1 Hz and
11 ./bhy sensor config -p /dev/ttyACM2 -sensor 10 -rate 1
12
13 # disable sensor 10
14 ./bhy sensor config -p /dev/ttyACM2 -sensor 10 -rate 1
```



/dev/ttyACM2 needs to be replaced with the (second) port from the host device retrieved through the list command.

First execute a list command to retrieve the available ports. The output should look similar to this:

```

1 $ ./bhy list
2 Found port: /dev/cu.usbmodem142301
3   USB ID    2341:8054
4   USB serial 554466C450534B54332E3120FF03071B554466
5 Found port: /dev/cu.usbmodem142303
6   USB ID    2341:8054
7   USB serial 554466C450534B54332E3120FF03071B554466
```

Then execute a config command to enable the desired sensor. Please refer to the [Sensor IDs](#) section to find the desired sensor ID. The output of this command should look similar to this:

```

1 $ ./bhy sensor config -p /dev/cu.usbmodem142303 -sen
2 Connected - port: /dev/cu.usbmodem142303 - baudrate:
3 Sending configuration: sensor 10 rate 1.000000
4 Sensor configuration correctly sent!

```

Then you can retrieve sensor data from the sensor using the read command. Here is an example output:

```

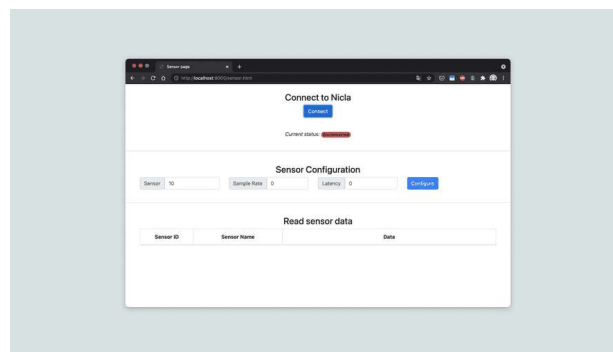
1 $ ./bhy sensor read -live -p /dev/cu.usbmodem142303
2 Connected - port: /dev/cu.usbmodem142303 - baudrate:
3 Sensor id: 10 name: GYRO_PASS values: x : -12.00
4 Sensor id: 10 name: GYRO_PASS values: x : -13.00

```

i If there is not ESLOV activity in the first minute after power up, the LDO is disabled and then also ESLOV is disabled. This has been made for low power reasons. However, you could change this timeout by calling `BHY2.setLDOTimeout(milliseconds)` from your sketch.

Sensor Data Over WebBLE

Sensor data from the Nicla Sense ME can also be retrieved through Bluetooth® Low Energy in the web browser. For that you can use the bhy tool. Please follow steps 1 - 3 from the "Sensor Data Over ESLOV" section. Then execute the following command to start the webserver: `./bhy webserver`. When the server has started, you can open the landing page in your browser: <http://localhost:8000/>. Click on "Open sensor page".

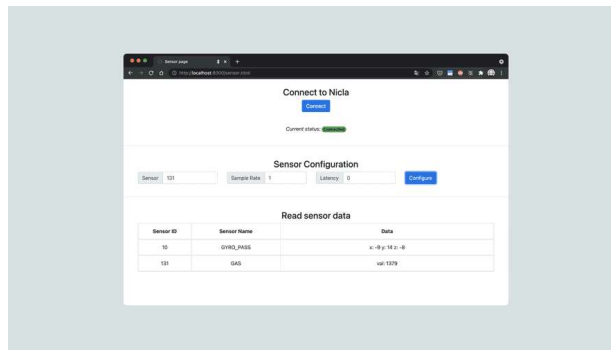


Sensor page in the browser

Then click the "Connect" button and pair your computer with the Nicla Sense ME board.

i For this feature to work, make sure that Web Bluetooth® Low Energy is both supported and enabled! In Google Chrome go to <chrome://flags> and enable "Experimental Web Platform features".

Use the sensor IDs from the section "Sensor IDs" to enable and configure the desired sensors. A sample rate > 0 will enable the sensor.



Configured sensors

BSX Sensor Fusion Software

The BHI260AP sensor runs a customizable firmware based on the BSX Sensor Fusion library. It provides a complete 9-axis fusion solution, which combines the measurements from 3-axis gyroscope, 3-axis geomagnetic sensor and a 3-axis accelerometer, to provide a robust absolute orientation vector. The algorithm fuses the sensor raw data from the accelerometer, geomagnetic sensor and gyroscope in an intelligent way to improve each sensor's output.

Go to this [site](#) or take a look at the BHI260AP's [datasheet](#) for more information.

Communication

Like other Arduino® products, the Nicla Sense ME features dedicated pins for different protocols.

SPI

The pins used for SPI (Serial Peripheral Interface) on the Nicla Sense ME are the following:

CS
CIPO
COPI
SCLK

You can refer to the [pinout](#) above to find them on the board.

To use SPI, you first need to include the [SPI](#) library.

```
1 #include <SPI.h>
```

Inside `void setup()` you need to initialize the library.

```
1 SPI.begin();
```

And to write to the device:

```
1 digitalWrite(chipSelectPin, LOW); //pull down the CS
2
3 SPI.transfer(address); // address for device, for
4 SPI.transfer(value); // value to write
5
6 digitalWrite(chipSelectPin, HIGH); // pull up the
```

I2C

The pins used for I2C (Inter-Integrated Circuit) on the Nicla Sense ME are the following:

SDA: SDA1

SCL: SCL1

You can refer to the [pinout](#) above to find them on the board.

To use I2C, you can use the [Wire](#) library, which you need to include at the top of your sketch.

```
1 #include <Wire.h>
```

Inside `void setup()` you need to initialize the library.

```
1 Wire.begin();
```

And to write something to a device connected via I2C, you can use the following commands:

```
1 Wire.beginTransmission(1); //begin transmit to device
2 Wire.write(byte(0x00)); //send instruction byte
3 Wire.write(val); //send a value
4 Wire.endTransmission(); //stop transmit
```

UART

The pins used for UART (Universal asynchronous receiver-transmitter) are the following:

Rx: GPIO2

Tx: GPIO1

You can refer to the pinout above to find them on the board.

To send and receive data through UART, you will first need to set the baud rate inside `void setup()`.

```
1 Serial1.begin(9600);
```

To read incoming data, you can use a while loop() to read each individual character and add it to a string.

```
1 while(Serial1.available()){
2     delay(2);
3     char c = Serial1.read();
4     incoming += c;
5 }
```

And to write something, you can use the following command:

```
1 Serial1.write("Hello world!");
```

Bluetooth® Low Energy

Using the BHY2Host Library with Bluetooth® Low Energy

The BHY2 library for the Nicla Sense ME can automatically send sensor values over a Bluetooth® Low Energy connection to a host board that uses the **Arduino_BHY2Host** library.

Include the BHY2Host library at the top of the sketch of the **Host** board

Configure the sensors the same way as you do with the BHY2 library except of the `begin` function that takes a `NICLA_VIA_BLE` parameter.

```

1  #include "Arduino BHY2Host.h"
2
3  Sensor temperature(SENSOR_ID_TEMP);
4  int lastCheck = 0;
5
6  void setup(){
7      Serial.begin(115200);
8      BHY2Host.begin(false, NICLA_VIA_BLE);
9      temperature.begin();
10 }
11
12 void loop(){
13     static auto lastCheck= millis();
14     BHY2Host.update();
15
16     // Check sensor values every second
17     if (millis() - lastCheck >= 1000) {
18         lastCheck = millis();
19         Serial.println(String("temperature: ") + String
20     }
21 }
```

The parameters of `BHY2Host::begin` are: data pass through and communication configuration. The first parameter defines if the data should be passed through the Serial connection. This allows to control the Nicla Sense ME from a PC when connected through a host board. You can use the `arduino-bhy` tool to control the Nicla Sense ME from either the PC command line or from a web page. The second parameter can take one of the following values: `NICLA_VIA_BLE`, `NICLA_AS_SHIELD`, `NICLA_VIA_ESLOV` (default).

Using the Bluetooth® Low Energy Library

To enable Bluetooth® Low Energy on the Nicla Sense ME, you can use the [ArduinoBLE](#) library. The example sketches included in the library work also with Nicla Sense ME with some minor modifications:

Include the Nicla System header at the top of your sketch:

```

1  #include "Nicla_System.h"
```

In the `setup()` function add:

```

1  nicla::begin();
```

Here is an example of how to use the Bluetooth® Low Energy library to advertise a byte characteristic that can be used for example to toggle an LED.

Include the library header it at the top of your sketch:

```
1 #include <ArduinoBLE.h>
```

Set the service and characteristic:

```
1 BLEService ledService("180A"); // BLE LED Service
2 BLEByteCharacteristic switchCharacteristic("2A57", B
```

Set advertised name and service:

```
1 BLE.setLocalName("Nicla Sense ME");
2 BLE.setAdvertisedService(ledService);
```

Start advertising:

```
1 BLE.advertise();
```

Listen for BLE peripherals to connect:

```
1 BLEDevice central = BLE.central();
```

Conclusion

This cheat sheet is written as a quick reference mainly to look up the features of this product. For a more in-depth walk through experience, please have a look at the other tutorials.

Suggest changes

The content on
docs.arduino.cc is
facilitated through a public

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is
licensed under the [Creative Commons Attribution-Share](#)

Was this article helpful?

